

**TME 4 : Orchestration de conteneurs – Introduction à  
Kubernetes**

HUANG Loïc 21203938  
HARRAOUI Ines 21204796

# Introduction et préparation :

## Introduction :

Commandes kubectl :

- **kubectl get nodes** : Affiche tous les nodes (machines) dans notre environnement.
- **kubectl get pods --all-namespaces** : Affiche les pods de tous les namespaces présents dans mon environnement.

```
root@debian:~# kubectl get nodes
NAME        STATUS    ROLES    AGE      VERSION
debian      Ready    control-plane,master   3m31s   v1.23.17
root@debian:~# kubectl get pods --all-namespaces
NAMESPACE   NAME                                     READY   STATUS    RESTARTS   AGE
kube-flannel kube-flannel-ds-jcjd                   1/1     Running   0           3m33s
kube-system coredns-bd6b6df9f-jcg77               1/1     Running   0           3m32s
kube-system coredns-bd6b6df9f-znxwm               1/1     Running   0           3m32s
kube-system etcd-debian                       1/1     Running   0           3m44s
kube-system kube-apiserver-debian        1/1     Running   0           3m44s
kube-system kube-controller-manager-debian 1/1     Running   0           3m45s
kube-system kube-proxy-gj9sg            1/1     Running   0           3m33s
kube-system kube-scheduler-debian        1/1     Running   0           3m44s
```

- **docker image list + docker container list** : la liste des images téléchargées et des conteneurs lancés pour kubernetes

```
root@debian:~# docker image list
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
ghcr.io/flannel-io/flannel      v0.26.4            1421c6bce6d6       12 months ago      76.9MB
ghcr.io/flannel-io/flannel-cni-plugin  v1.6.2-flannell  55ce2385d9d8       12 months ago      10.7MB
registry.k8s.io/metrics-server/metrics-server  v0.7.2            48d9cfaaf390       17 months ago      67.1MB
registry.k8s.io/kube-apiserver  v1.23.17          62bc5d8258d6       2 years ago        130MB
registry.k8s.io/kube-controller-manager  v1.23.17          1dab4fc7b6e0       2 years ago        120MB
registry.k8s.io/kube-proxy      v1.23.17          f21c8d21558c       2 years ago        111MB
registry.k8s.io/kube-scheduler  v1.23.17          bc6794cb54ac       2 years ago        51.9MB
docker/getting-started          latest            3e4394f6b72f       3 years ago        47MB
registry.k8s.io/etcd            3.5.6-0          fce326961ae2       3 years ago        299MB
k8s.gcr.io/metrics-server/metrics-server  v0.5.2            f73640fb5061       4 years ago        64.3MB
registry.k8s.io/coredns/coredns  v1.8.6            a4ca41631cc7       4 years ago        46.8MB
registry.k8s.io/pause           3.6               6270bb605e12       4 years ago        683KB
imunes/template                 latest            c009531c75fd       6 years ago        326MB

root@debian:~# docker container list
CONTAINER ID        IMAGE               COMMAND                  CREATED             STATUS              PORTS              NAMES
71a2ae04c631       a4ca41631cc7       "/coredns -conf /etc..." 3 minutes ago      Up 3 minutes                k8s_coredns_core
ed8f93d84452       a4ca41631cc7       "/coredns -conf /etc..." 3 minutes ago      Up 3 minutes                k8s_coredns_core
4bc424d20864       registry.k8s.io/pause:3.6   "/pause"                 3 minutes ago      Up 3 minutes                k8s_POD_coredns-
bd6b6df9f-jcg77_kube-system_2018077b-fda5-4a15-a079-9292697aebcc_0
e4046b065387       registry.k8s.io/pause:3.6   "/pause"                 3 minutes ago      Up 3 minutes                k8s_POD_coredns-
bd6b6df9f-jcg77_kube-system_2018077b-fda5-4a15-a079-9292697aebcc_0
d47e68503c43       1421c6bce6d6       "/opt/bin/flanneld -..." 3 minutes ago      Up 3 minutes                k8s_kube-flannel
kube-flannel-ds-jcjd_kube-flannel_fd8cd6ee-dccc-4ce4-aeb3-14a71feb2e2c_0
4c7d514d4e14       f21c8d21558c       "/usr/local/bin/kube..." 3 minutes ago      Up 3 minutes                k8s_kube-proxy_k
ube-proxy-gj9sg_kube-system_fb464bfa-11bc-42ec-bfc3-d2d8c31a075b_0
dd77f60d1080       registry.k8s.io/pause:3.6   "/pause"                 3 minutes ago      Up 3 minutes                k8s_POD_kube-fla
nnel-ds-jcjd_kube-flannel_fd8cd6ee-dccc-4ce4-aeb3-14a71feb2e2c_0
79b4fae5228       registry.k8s.io/pause:3.6   "/pause"                 3 minutes ago      Up 3 minutes                k8s_POD_kube-pro
xy-gj9sg_kube-system_fb464bfa-11bc-42ec-bfc3-d2d8c31a075b_0
5ee90551bafb       62bc5d8258d6       "kube-apiserver --ad..." 4 minutes ago      Up 4 minutes                k8s_kube-apiserv
```

## Question 1 :

Création d'un fichier de configuration de déploiement yaml pour déployer 5 répliques du serveur web de l'image docker getting-started. Remarque : Ce fichier de configuration est appelé aussi *blueprint* ou *manifeste*. La commande **kubectl apply -f myserver-deployment.yaml** : crée ou met à jour le déploiement dans Kubernetes.

Fichier de configuration de déploiement (YAML) :

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myserver-deployment
  labels:
    app: myserver
spec:
  replicas: 5
  selector:
    matchLabels:
      app: myserver
  template:
    metadata:
      labels:
        app: myserver
    spec:
      containers:
        - name: myserver
          image: docker/getting-started:latest
          imagePullPolicy: Never
          ports:
            - containerPort: 80
```

Vérification du déploiement :

```
root@debian:~# kubectl get deployment
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
myserver-deployment 5/5      5             5           38m
root@debian:~# kubectl get pod
NAME                READY   STATUS    RESTARTS   AGE
myserver-deployment-765ff778fd-dbwrk 1/1     Running   0           52s
myserver-deployment-765ff778fd-fqcwq 1/1     Running   0           52s
myserver-deployment-765ff778fd-psc26 1/1     Running   0          2m34s
myserver-deployment-765ff778fd-s8vzn 1/1     Running   0          2m36s
myserver-deployment-765ff778fd-sbxzz 1/1     Running   0           52s
root@debian:~# kubectl get pod -o wide
NAME                READY   STATUS    RESTARTS   AGE   IP            NODE    NOMINATED NODE   READ
INESS GATES
myserver-deployment-765ff778fd-dbwrk 1/1     Running   0           57s   10.244.0.14   debian  <none>            <non
e>
myserver-deployment-765ff778fd-fqcwq 1/1     Running   0           57s   10.244.0.15   debian  <none>            <non
e>
myserver-deployment-765ff778fd-psc26 1/1     Running   0          2m39s   10.244.0.12   debian  <none>            <non
e>
myserver-deployment-765ff778fd-s8vzn 1/1     Running   0          2m41s   10.244.0.11   debian  <none>            <non
e>
myserver-deployment-765ff778fd-sbxzz 1/1     Running   0           57s   10.244.0.13   debian  <none>            <non
e>
root@debian:~# docker container ps | grep myserver
64dc216caeb0      3e4394f6b72f      "/docker-entrypoint..." About a minute ago Up About a minute
k8s_myserver_myserver-deployment-765ff778fd-fqcwq_default_1e0100da-27b9-4244-a07d-6459b360ab1a_0
5a7bbfa5347e      3e4394f6b72f      "/docker-entrypoint..." About a minute ago Up About a minute
k8s_myserver_myserver-deployment-765ff778fd-dbwrk_default_149dc4dd-95f0-401e-8e7e-5f88d8bb3cfb_0
ff7d1b6029a7      3e4394f6b72f      "/docker-entrypoint..." About a minute ago Up About a minute
k8s_myserver_myserver-deployment-765ff778fd-sbxzz_default_d70ae242-fe8a-4101-a485-4feba6604bae_0
e5f2acef19f2      registry.k8s.io/pause:3.6 "/pause" About a minute ago Up About a minute
k8s_POD_myserver-deployment-765ff778fd-fqcwq_default_1e0100da-27b9-4244-a07d-6459b360ab1a_0
b43ce374e1b2      registry.k8s.io/pause:3.6 "/pause" About a minute ago Up About a minute
k8s_POD_myserver-deployment-765ff778fd-sbxzz_default_d70ae242-fe8a-4101-a485-4feba6604bae_0
55012b5af81a      registry.k8s.io/pause:3.6 "/pause" About a minute ago Up About a minute
k8s_POD_myserver-deployment-765ff778fd-dbwrk_default_149dc4dd-95f0-401e-8e7e-5f88d8bb3cfb_0
7d30c963e5ee      3e4394f6b72f      "/docker-entrypoint..." 2 minutes ago Up 2 minutes
k8s_myserver_myserver-deployment-765ff778fd-psc26_default_9490164a-7e4c-4479-81e2-f77d886166fd_0
586f1f665461      registry.k8s.io/pause:3.6 "/pause" 2 minutes ago Up 2 minutes
k8s_POD_myserver-deployment-765ff778fd-psc26_default_9490164a-7e4c-4479-81e2-f77d886166fd_0
db7a7e3ed6a4      3e4394f6b72f      "/docker-entrypoint..." 2 minutes ago Up 2 minutes
k8s_myserver_myserver-deployment-765ff778fd-s8vzn_default_73260ce1-abce-41e0-b8bf-9f6b89f1ae24_0
23cf2678ddd8      registry.k8s.io/pause:3.6 "/pause" 2 minutes ago Up 2 minutes
k8s_POD_myserver-deployment-765ff778fd-s8vzn_default_73260ce1-abce-41e0-b8bf-9f6b89f1ae24_0
```

## 2/ Auto-réparation ('self-healing')

Question 2:

Nous allons forcer l'arrêt d'un des conteneurs avec les commandes utilisées sur l'image ci-dessous. Cependant la réparation étant très rapide, nous n'avons pas pu voir 0/1 dans la colonne READY. Néanmoins, on a pu constater qu'il a bien été réparé grâce à la colonne RESTARTS.

```
root@debian:~# docker container list | grep myserver-deployment-765ff778fd-fqcwq
64dc216caeb0      3e4394f6b72f      "/docker-entrypoint..." 4 minutes ago      Up 4 minutes
k8s_myserver_myserver-deployment-765ff778fd-fqcwq_default_1e0100da-27b9-4244-a07d-6459b360ab1a_0
e5f2acef19f2      registry.k8s.io/pause:3.6  "/pause"                  4 minutes ago      Up 4 minutes
k8s_POD_myserver-deployment-765ff778fd-fqcwq_default_1e0100da-27b9-4244-a07d-6459b360ab1a_0
root@debian:~# kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
myserver-deployment-765ff778fd-dbwrk 1/1     Running   0           5m8s
myserver-deployment-765ff778fd-fqcwq 1/1     Running   0           5m8s
myserver-deployment-765ff778fd-psc26 1/1     Running   0           6m50s
myserver-deployment-765ff778fd-s8vnz 1/1     Running   0           6m52s
myserver-deployment-765ff778fd-sbxzz 1/1     Running   0           5m8s
root@debian:~# docker container stop 64dc216caeb0 e5f2acef19f2
64dc216caeb0
e5f2acef19f2
root@debian:~# kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
myserver-deployment-765ff778fd-dbwrk 1/1     Running   0           5m48s
myserver-deployment-765ff778fd-fqcwq 1/1     Running   1 (6s ago)  5m48s
myserver-deployment-765ff778fd-psc26 1/1     Running   0           7m30s
myserver-deployment-765ff778fd-s8vnz 1/1     Running   0           7m32s
myserver-deployment-765ff778fd-sbxzz 1/1     Running   0           5m48s
```

Commande `KUBE_EDITOR="gedit" kubectl edit deployment/myserver-deployment -o yaml`:

```
status:
  availableReplicas: 5
  conditions:
  - lastTransitionTime: "2026-02-16T13:45:47Z"
    lastUpdateTime: "2026-02-16T13:47:37Z"
    message: ReplicaSet "myserver-deployment-765ff778fd" has successfully progressed.
    reason: NewReplicaSetAvailable
    status: "True"
    type: Progressing
  - lastTransitionTime: "2026-02-16T13:49:20Z"
    lastUpdateTime: "2026-02-16T13:49:20Z"
    message: Deployment has minimum availability.
    reason: MinimumReplicasAvailable
    status: "True"
    type: Available
  observedGeneration: 4
  readyReplicas: 5
  replicas: 5
  updatedReplicas: 5
```

La section status d'un déploiement Kubernetes indique les **messages d'erreurs obtenus lors du déploiement**. Il est maintenu en interne ce qui permet entre autres de vérifier l'état des conteneurs et comparer entre l'état actuel et l'état désiré.

### Question 3:

Méthode 1 : `kubectl expose deployment myserver-deployment --target-port=80 --port=12345 --name=myfirstservice`

Exemple :

```
root@debian:~# kubectl get services
NAME                TYPE          CLUSTER-IP      EXTERNAL-IP  PORT(S)          AGE
kubernetes          ClusterIP     10.96.0.1        <none>       443/TCP          95m
myserver-deployment ClusterIP     10.100.247.163   <none>       12345/TCP        33m
root@debian:~# kubectl delete service myserver-deployment
service "myserver-deployment" deleted
root@debian:~# kubectl expose deployment myserver-deployment --target-port=80 --port=12345 --name=myfirst
service/myfirstservice exposed
root@debian:~# kubectl get services
NAME                TYPE          CLUSTER-IP      EXTERNAL-IP  PORT(S)          AGE
kubernetes          ClusterIP     10.96.0.1        <none>       443/TCP          96m
myfirstservice      ClusterIP     10.107.123.178   <none>       12345/TCP        4s
root@debian:~# kubectl describe services myfirstservice
Name:                myfirstservice
Namespace:           default
Labels:              app=myserver
Annotations:         <none>
Selector:            app=myserver
Type:               ClusterIP
IP Family Policy:    SingleStack
IP Families:         IPv4
IP:                 10.107.123.178
IPs:                10.107.123.178
Port:               <unset> 12345/TCP
TargetPort:         80/TCP
Endpoints:          10.244.0.11:80,10.244.0.12:80,10.244.0.13:80 + 2 more...
Session Affinity:   None
Events:             <none>
```

Commandes :

- `ip link show type bridge` : Affiche la liste des interfaces de type bridge avec leur informations
- `ip a s cni0 (s = show)` : Affiche les informations de l'interface cni0
- `ip link show type vxlan` : Affiche la liste des interfaces de type vxlan
- `/opt/cni/bin/flannel` : Affiche les caractéristiques du plugin réseaux.

```
root@debian:~# ip link show type bridge
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN mode DEFAULT group default
   link/ether 02:42:fc:e1:35:88 brd ff:ff:ff:ff:ff:ff
5: cni0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1450 qdisc noqueue state UP mode DEFAULT group default qlen 1000
   link/ether 76:72:b0:68:be:de brd ff:ff:ff:ff:ff:ff
root@debian:~# ip a s cni0
5: cni0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1450 qdisc noqueue state UP group default qlen 1000
   link/ether 76:72:b0:68:be:de brd ff:ff:ff:ff:ff:ff
   inet 10.244.0.1/24 brd 10.244.0.255 scope global cni0
       valid_lft forever preferred_lft forever
   inet6 fe80::7472:b0ff:fe68:bede/64 scope link
       valid_lft forever preferred_lft forever
root@debian:~# ip link show type vxlan
4: flannel.1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1450 qdisc noqueue state UNKNOWN mode DEFAULT group default
   link/ether 0a:3e:e1:27:83:6b brd ff:ff:ff:ff:ff:ff
root@debian:~# /opt/cni/bin/flannel
CNI Plugin flannel version v1.0.0-dev+.dirty (linux/amd64) commit .dirty built on 2025-01-31T08:53:49Z
CNI protocol versions supported: 0.1.0, 0.2.0, 0.3.0, 0.3.1, 0.4.0, 1.0.0, 1.1.0
```

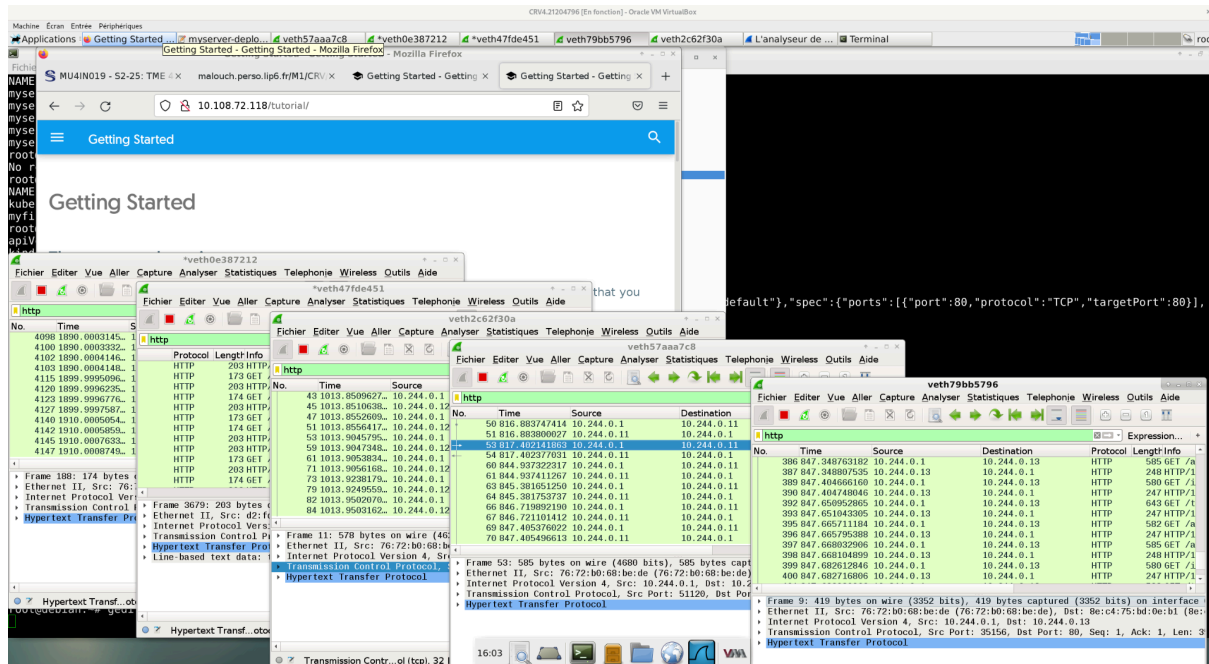
Le pont linux cni0 a été créé afin de connecter les pods dans le nœud worker grâce au plugin flannel à la demande de kubernetes selon les spécifications CNI, d'où le nom cni0.

### Quel est le rôle de l'interface flannel.1 ?

Le rôle de flannel.1 est de pouvoir permettre aux pods de différents nœuds de communiquer entre eux. (VXLAN permet de créer un réseau de niveau 2 sur un réseau de niveau 3.)

Quel(s) pod(s) répond(ent) aux requêtes http ? Décrivez globalement les résultats des rechargements de la page.

Tous les pods répondent aux requêtes http. A chaque reload de la page, le service choisit aléatoirement un pod pour répondre à la requête HTTP.



Méthode 2 : Création d'un service depuis le fichier de configuration de déploiement

Fichier de configuration de déploiement (YAML) :

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myserver-deployment
  labels:
    app: myserver
spec:
  selector:
    matchLabels:
      app: myserver
  template:
    metadata:
      labels:
        app: myserver
    spec:
      containers:
        - name: myserver
          image: docker/getting-started:latest
          imagePullPolicy: Never
          ports:
            - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: myfirstservice
spec:
  selector:
    app: myserver
  ports:
    - protocol: TCP
apiVersion: autoscaling/v2
port: 80
targetPort: 9376
```

Quel est le type de votre service ? (Utilisez la commande **kubectl get services myfirstservice -o yaml**)

Le type de notre service est ClusterIP qui n'est identique qu'avec la méthode 1.

Explication de chaque ligne du code YAML implémentant kind Service :

- **kind : Service** : L'objet qu'on veut implémenter dans Kubernetes. Ici un service.
- **metadata** : cette section contient les informations qui permettent d'identifier l'objet qu'on va créer, qui n'est pas utile pour le bon fonctionnement du service.
  - **name : myfirstservice** : C'est le nom qu'on a donné au service.
- **spec** : La section spec, contient les spécifications du service.
  - **selector** : Indique au service quel pod va gérer ce qu'il transmet.
    - **app : myserver** : Tous les pods qui ont un label (app : myserver) seront ceux qui vont traiter les informations que transmet le service.
  - **ports** : Indique toutes les caractéristiques réseaux que le service va utiliser.
    - **protocol : TCP** : Le protocole que le service utilise.
    - **port : 80** : Le numéro du port utilisé par le service.
    - **targetport : 9376** : Le port utilisé par les pods.

### 3/ La remise à l'échelle horizontale automatique des pods ('Horizontal Pod Auto-scaling')

Question 3.1 :

La valeur de l'utilisation courante affichée par **kubectl get hpa** est bien égale à la valeur CPU(cores) affichée par **kubectl top pods** divisée par la demande cpu de 15 (1/15 = 0.06 => 6%). Le nombre de pods (replicas) n'a pas augmenté parce qu'on n'a pas encore atteint le seuil qu'on a défini avec HPA. Le seuil qu'on a défini est de 20%, si nous sommes en dessous du seuil, il n'y a pas besoin d'ajouter de pod.

```
root@debian:~# kubectl get hpa
NAME      REFERENCE                      TARGETS  MINPODS  MAXPODS  REPLICAS  AGE
hpa       Deployment/myserver-deployment  6%/20%   1         20        1         35m
root@debian:~# kubectl top pods
NAME                                CPU(cores)  MEMORY(bytes)
myserver-deployment-f75695695-6tjjq  1m          5Mi
```

Question 3.2 :

Avec la diminution de la valeur du sleep, passant de 0.5 à 0.2, on remarque que l'utilisation du CPU par le pod est passée à 13%.

```
root@debian:~# kubectl get hpa --watch
NAME      REFERENCE                      TARGETS  MINPODS  MAXPODS  REPLICAS  AGE
hpa       Deployment/myserver-deployment  0%/20%   1         20        1         22m
hpa       Deployment/myserver-deployment  6%/20%   1         20        1         33m
hpa       Deployment/myserver-deployment  0%/20%   1         20        1         49m
hpa       Deployment/myserver-deployment  6%/20%   1         20        1         50m
hpa       Deployment/myserver-deployment  13%/20%  1         20        1         50m
```



### Question 3.3 :

Avec la diminution de la valeur du sleep, passant de 0.2 à 0.05, on remarque que l'utilisation du CPU par le pod dépasse le seuil des 20%. Ce qui déclenche l'ajout d'un pod identique au pod de la configuration afin de répartir la tâche aux deux pods à la place d'une. La valeur de la target sera alors la moyenne d'utilisation du CPU par node. 16% d'utilisation du CPU par les 2 pods.

```
root@debian:~# kubectl get hpa --watch
```

| NAME | REFERENCE                      | TARGETS | MINPODS | MAXPODS | REPLICAS | AGE |
|------|--------------------------------|---------|---------|---------|----------|-----|
| hpa  | Deployment/myserver-deployment | 0%/20%  | 1       | 20      | 1        | 22m |
| hpa  | Deployment/myserver-deployment | 6%/20%  | 1       | 20      | 1        | 33m |
| hpa  | Deployment/myserver-deployment | 0%/20%  | 1       | 20      | 1        | 49m |
| hpa  | Deployment/myserver-deployment | 6%/20%  | 1       | 20      | 1        | 50m |
| hpa  | Deployment/myserver-deployment | 13%/20% | 1       | 20      | 1        | 50m |
| hpa  | Deployment/myserver-deployment | 33%/20% | 1       | 20      | 1        | 64m |
| hpa  | Deployment/myserver-deployment | 33%/20% | 1       | 20      | 2        | 65m |
| hpa  | Deployment/myserver-deployment | 26%/20% | 1       | 20      | 2        | 65m |
| hpa  | Deployment/myserver-deployment | 20%/20% | 1       | 20      | 2        | 65m |
| hpa  | Deployment/myserver-deployment | 16%/20% | 1       | 20      | 2        | 66m |

### Question 3.4 :

Avec la diminution de la valeur du sleep, passant de 0.05 à 0.01, on remarque que l'utilisation du CPU par les pods dépasse le seuil des 20%. Ce qui déclenche une nouvelle fois l'ajout de pod. HPA va d'abord ajouter 2 pods car chaque pod a une utilisation de 40% du CPU. Ensuite il va rajouter 1 pod pour avoir une moyenne d'utilisation du CPU par pod inférieur ou égale à 20%.

```
root@debian:~# kubectl get hpa --watch
```

| NAME | REFERENCE                      | TARGETS | MINPODS | MAXPODS | REPLICAS | AGE |
|------|--------------------------------|---------|---------|---------|----------|-----|
| hpa  | Deployment/myserver-deployment | 0%/20%  | 1       | 20      | 1        | 22m |
| hpa  | Deployment/myserver-deployment | 6%/20%  | 1       | 20      | 1        | 33m |
| hpa  | Deployment/myserver-deployment | 0%/20%  | 1       | 20      | 1        | 49m |
| hpa  | Deployment/myserver-deployment | 6%/20%  | 1       | 20      | 1        | 50m |
| hpa  | Deployment/myserver-deployment | 13%/20% | 1       | 20      | 1        | 50m |
| hpa  | Deployment/myserver-deployment | 33%/20% | 1       | 20      | 1        | 64m |
| hpa  | Deployment/myserver-deployment | 33%/20% | 1       | 20      | 2        | 65m |
| hpa  | Deployment/myserver-deployment | 26%/20% | 1       | 20      | 2        | 65m |
| hpa  | Deployment/myserver-deployment | 20%/20% | 1       | 20      | 2        | 65m |
| hpa  | Deployment/myserver-deployment | 16%/20% | 1       | 20      | 2        | 66m |
| hpa  | Deployment/myserver-deployment | 20%/20% | 1       | 20      | 2        | 66m |
| hpa  | Deployment/myserver-deployment | 10%/20% | 1       | 20      | 2        | 67m |
| hpa  | Deployment/myserver-deployment | 20%/20% | 1       | 20      | 2        | 67m |
| hpa  | Deployment/myserver-deployment | 43%/20% | 1       | 20      | 2        | 67m |
| hpa  | Deployment/myserver-deployment | 46%/20% | 1       | 20      | 4        | 67m |
| hpa  | Deployment/myserver-deployment | 36%/20% | 1       | 20      | 5        | 68m |

```
root@debian:~# kubectl get hpa --watch
```

| NAME | REFERENCE                      | TARGETS | MINPODS | MAXPODS | REPLICAS | AGE |
|------|--------------------------------|---------|---------|---------|----------|-----|
| hpa  | Deployment/myserver-deployment | 20%/20% | 1       | 20      | 5        | 68m |



### Question 3.5 :

A l'arrêt de la commande **for (;;)** **do wget**, on observe que l'utilisation du CPU par les pods atteint les 0%. HPA va petit à petit réduire les pods parce qu'il va voir que la moyenne d'utilisation du CPU par pod est inférieure au seuil. Pour à la fin revenir à 1 pod.

| NAME | REFERENCE                      | TARGETS | MINPODS | MAXPODS | REPLICAS | AGE |
|------|--------------------------------|---------|---------|---------|----------|-----|
| hpa  | Deployment/myserver-deployment | 20%/20% | 1       | 20      | 5        | 68m |
| hpa  | Deployment/myserver-deployment | 9%/20%  | 1       | 20      | 5        | 73m |
| hpa  | Deployment/myserver-deployment | 1%/20%  | 1       | 20      | 5        | 73m |
| hpa  | Deployment/myserver-deployment | 0%/20%  | 1       | 20      | 5        | 74m |
| hpa  | Deployment/myserver-deployment | 0%/20%  | 1       | 20      | 5        | 78m |
| hpa  | Deployment/myserver-deployment | 0%/20%  | 1       | 20      | 3        | 78m |
| hpa  | Deployment/myserver-deployment | 0%/20%  | 1       | 20      | 1        | 78m |

Code YAML implémentant kind HPA:

```
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: myserver-deployment
  minReplicas: 1
  maxReplicas: 20
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 20
```

Explication de chaque ligne du code YAML implémentant kind HPA :

- **kind: HorizontalPodAutoscaler** : L'objet qu'on veut implémenter dans Kubernetes. Ici un HPA qui ajuste automatiquement le nombre de pods (réplicas) en fonction de la charge horizontale automatique des pods en fonction des métriques (CPU, mémoire, etc...). ('Horizontal Pod Auto-scaling')
- **metadata** : La section métadonnées contient les informations qui permet d'identifier l'objet qu'on va créer, qui n'est pas utile pour le bon fonctionnement du service. Exemple : son nom unique dans le namespace.
  - **name: hpa** : Nom de l'HPA dans Kubernetes, utilisé pour le référencer (kubectl get hpa name).
- **spec** : Spécification détaillée du comportement de l'HPA.
  - **scaleTargetRef** : Référence à l'objet à scaler (ici un Deployment)
    - **apiVersion: apps/v1** : Version API du Deployment cible

- **kind: Deployment** : Type d'objet Kubernetes que HPA va scaler (Deployment, ReplicaSet, etc.).
  - **name: myserver-deployment** : Nom exact du Deployment qui sera scalé par cet HPA.
- 
- **minReplicas: 1** : Nombre minimum de pods que l'HPA maintiendra
  - **maxReplicas: 20** : Maximum de pods que l'HPA peut créer en cas de surcharge
  - **metrics** : Liste des métriques surveillées pour déclencher le scaling (CPU, mémoire,...).
    - **type: Resource** : La métrique qu'on va regarder est une ressource du système.
    - **resource** : Section qui détaille la ressource qu'on va regarder.
      - **name: cpu** : Métrique basée sur le CPU.
      - **target**: Définit la condition de scaling pour cette métrique CPU
        - **type: Utilization** : Regarde l'utilisation du CPU (pourcentage d'utilisation)
        - **averageUtilization: 20** : Le seuil dans lequel on augmente le nombre de pods si l'utilisation CPU moyenne des pods dépasse les 20% pour bien répartir la charge et éviter la surcharge .